

Introduction

This article serves as an introduction to some fundamental concepts in binary number manipulation, including binary number conversion, one's complement, and two's complement and many more concepts. These concepts are not only fundamental to understanding how computers work but also crucial for solving a variety of problems in computer science.

Having an understanding of binary numbers is extremely important in the field of computer science as everything stored in a computer is in the form of only 0s and 1s.

Binary Number Conversion

Decimal to Binary Conversion:

By repeatedly dividing a number by 2 and recording the result, decimal values can be transformed into binary.

Example: Converting 13 to its binary equivalent:

- Start with the decimal number 13.
- Divide the number by 2 and record the remainder.
- Repeat the division with the quotient until the number becomes 0.

13 / 2 = 6 remainder 1
6 / 2 = 3 remainder 0
3 / 2 = 1 remainder 1
1 / 2 = 0 remainder 1

To obtain the binary equivalent of 13, read the remainders from bottom to top: **1101**.

So, the binary equivalent of 13 is **1101**.

Binary to Decimal Conversion:

Converting a binary number back to its decimal equivalent involves a reverse process.

Example: Converting 1101 to its decimal equivalent:

- Start from the rightmost bit (least significant bit).
- Each bit is multiplied by 2 raised to the power of its position index.

1 * 2^0 = 1
0 * 2^1 = 0
1 * 2^2 = 4
1 * 2^3 = 8

Sum = 1 + 0 + 4 + 8 = **13**.

Hence, the decimal equivalent of the binary number **1101** is **13**.

Understanding One's Complement and Two's Complement

One's Complement

The one's complement of a binary number is obtained by flipping all the bits.

Example: The one's complement of 13 (binary 1101):

Binary of 13 : 0000 1101
One's Complement : 1111 0010

Two's Complement

The two's complement is obtained by taking the one's complement of a number and adding 1.

Example: The two's complement of 13 (binary 1101):

One's Complement : 1111 0010
Add 1 : 1111 0011

Bitwise Operators

AND Operator (&)

If both corresponding bits are 1, the resulting bit is 1; otherwise, it is 0.

```
13: 1101
 7: 0111
&  : 0101 → 5
```

OR Operator (|)

If either corresponding bit is 1, the resulting bit is 1.

```
13: 1101
 7: 0111
|   : 1111 → 15
```

XOR Operator (^)

If bits differ, the result is 1; if the same, result is 0.

```
13: 1101
 7: 0111
^   : 1010 → 10
```

NOT Operator (~)

Flips all bits of the number.

```
5: 0000 0101
~5: 1111 1010 → -6 (in two's complement)
```

Shift Operators

Right Shift (>>): Shifts bits to the right, fills left with 0s.

```
13 >> 1 = 0110 → 6
```

Left Shift (<<): Shifts bits to the left, fills right with 0s.

```
13 << 1 = 11010 → 26
```

Bit Manipulation Tricks and Techniques

1. Swapping Two Numbers Without a Third Variable

```
A = A ^ B
B = A ^ B
A = A ^ B
```

2. Checking if the i-th Bit is Set

```
(1 << i) & num    → set if result ≠ 0
(num >> i) & 1    → set if result ≠ 0
```

3. Setting the i-th Bit

```
num | (1 << i)
```

4. Clearing the i-th Bit

```
num & ~(1 << i)
```

5. Toggling the i-th Bit

```
num ^ (1 << i)
```